MAPLE: Manifold-Activated Pruning of Latent Experts

Empirical Activation-Guided Expert Distillation for
Mixture-of-Experts Language Models Below 2 Bits Per Weight

Aman Sachan
Independent Systems Research
New Delhi, India
amansachan92905@gmail.com

November 2026

Abstract

We present MAPLE (**M**anifold-**A**ctivated **P**runing of **L**atent **E**xperts), a method for compressing Mixture-of-Experts (MoE) large language models to sub-2-bit-per-weight effective densities while retaining high functional quality on targeted deployment domains. The key insight is that in a fine-grained MoE model trained on a broad multilingual corpus, the vast majority of routed expert capacity is dormant for any given deployment subdomain. We exploit this structural sparsity by profiling the actual expert activation manifold during forward passes on a representative calibration corpus, scoring each expert by the product of its empirical call count and the square root of its mean input energy, and constructing a reduced GGUF container retaining only the top-k experts per layer. Applied to Gemma 4 26B-A4B (a 128-expert, 30-layer MoE at 9.92 GB in IQ2_XXS quantization), MAPLE produces a 4.75 GB model (1.48 effective bits per weight, 2.09× compression) that runs at 45.4 tokens per second with 100% GPU residency on a consumer RTX 4050 6GB laptop GPU—a 6.57× throughput improvement. Task-based semantic fidelity benchmarking across four diverse domains yields a mean keyword-match retention of 93.8%. We also document three failure modes that our method overcomes: proxy-embedding routing estimation, cascaded scale suppression, and sampling-hyperparameter interference with code generation. All code, calibration corpora, and expert selection maps are released openly.

1 Introduction

The proliferation of Mixture-of-Experts architectures in frontier large language models [8, 2, 18] has introduced a structurally novel compression opportunity that conventional quantization methods fail to exploit. In a dense transformer, every weight participates in every forward pass; compression therefore operates uniformly across the weight space. In a MoE model, however, only a small fraction of the total expert capacity is activated per token—typically k experts out of E total, with $k/E \approx 0.03$–0.06 for modern fine-grained architectures. The question this paper asks is deceptively simple: *for a specific deployment subdomain, which k' out of E experts ever matter—and can we safely discard the rest entirely?*

The practical stakes are high. Consumer-grade GPUs have VRAM ceilings of 6–12 GB. The difference between a model that *fits entirely in VRAM* and one that *must page weights through CPU RAM* is not a minor quality difference; it is a qualitative cliff. In our empirical setting, the full-precision-routed Gemma 4 26B-A4B at 9.92 GB runs at 6.9 tokens per second with CPU paging on an RTX 4050 6GB. After MAPLE compression to 4.75 GB it runs at 45.4 tokens per second entirely within VRAM. The user experience—from barely interactive to real-time conversational—is transformed.

Prior work on MoE compression has largely focused on weight quantization [5, 11], structured pruning of attention heads [14], and static expert selection based on router logits [9]. What is missing is a *forward-pass empirical* approach that uses the model's own observed activation statistics, computed on a representative calibration corpus, to decide which experts survive compression. This is precisely what MAPLE provides.

This paper makes the following contributions:

1. We introduce the *empirical activation manifold* of a MoE model: the per-layer distribution of expert call counts and input energy statistics, obtained through native forward-pass profiling (§4).
2. We define the MAPLE expert scoring function $s_e = c_e \cdot \sqrt{\bar{\sigma}_e}$, where c_e is the empirical call count and $\bar{\sigma}_e$ is the mean input L2 energy of expert e, and show it outperforms count-only or energy-only selection (§4).
3. We document three distinct failure modes that invalidate naive compression approaches and explain precisely why each fails (§5).
4. We demonstrate 93.8% task fidelity retention, 2.09× compression, 1.48 effective BPW, and 6.57× throughput improvement on a commodity laptop GPU for a 26-billion-parameter MoE model (§6).
5. We release all code, calibration data, and expert selection maps at the project repository.

The paper is organized as follows. Section 2 reviews MoE architectures and existing compression approaches. Section 3 characterizes the expert sparsity structure we exploit. Section 4 describes MAPLE in full algorithmic detail. Section 5 documents the three failure modes. Section 6 presents quantitative evaluation. Section 7 discusses limitations honestly. Section 8 addresses generalizability. Section 9 situates our work in context. Section 10 concludes.

2 Background

2.1 Mixture-of-Experts Language Models

A Mixture-of-Experts transformer replaces the dense feed-forward network (FFN) in each transformer block with a set of E parallel expert networks and a learned router. For each input token hidden state $\mathbf{h} \in \mathbb{R}^d$, the router computes logits $\mathbf{g} = \mathbf{h}W_{\text{gate}} \in \mathbb{R}^E$, selects the top-$k$ experts by logit magnitude, applies a softmax-normalized weighting over the selected experts, and sums their outputs:

$$\text{FFN}_{\text{MoE}}(\mathbf{h}) = \sum_{e \in \text{top}_k(\mathbf{g})} \text{softmax}(\mathbf{g})_e \cdot \text{FFN}_e(\mathbf{h}), \tag{1}$$

where each FFN_e is a standard two-layer feed-forward network with gate, up-projection, and down-projection matrices.

Fine-grained MoE architectures [2] use large E (64–256 experts) with small k (4–8 active), so individual experts are narrow specialists. The total parameter count scales with E, but the per-token compute cost scales only with k. Gemma 4 26B-A4B [18] uses $E = 128$, $k = 8$, and 30 transformer blocks in which every block is a MoE block (additionally containing a shared dense FFN branch).

2.2 Existing Compression Approaches

Post-training quantization (PTQ). Methods such as GPTQ [5] and AWQ [11] minimize quantization error on calibration data by applying second-order weight updates (GPTQ) or activation-aware scaling (AWQ). They reduce precision uniformly across weights but do not reduce the number of experts; memory and compute reductions are proportional to the bit-width reduction alone.

Importance-based structured pruning. L0 regularization [13], movement pruning [15], and magnitude-based pruning [7] prune weight tensors or entire components (heads, layers). Applied to MoE models, structured pruning could remove entire expert networks; however, existing implementations typically require gradient access to the original training or fine-tuning pipeline, which is not available for community-distributed GGUF models.

Expert routing analysis. Several works have analyzed router behavior to understand load balancing [10] and expert specialization [4]. However, these works operate in the training regime with full gradient access. The closest antecedents to our work are expert dropping methods [1], which analyze expert importance based on routing probabilities, but do so using aggregated training statistics rather than forward-pass activation profiles on deployment calibration data.

GGUF quantization. The GGUF format [12] used by llama.cpp supports mixed-precision quantization schemes (Q4_K, Q5_K, IQ2_XXS, etc.) and an importance matrix (`imatrix`) mechanism that records per-weight activation importance for calibration-aware quantization. We repurpose this `imatrix` infrastructure for our expert selection step, treating it as a lightweight profiling instrument rather than a quantization calibration tool.

3 The Core Insight: Expert Sparsity in Deployment Domains

A MoE model trained on a global multilingual corpus must route diverse linguistic phenomena—code in dozens of programming languages, mathematics in multiple notations, natural language across hundreds of languages, and specialized domain text—across its expert pool. During training, the router learns to assign tokens from different distributions to different experts, so the full expert population is collectively necessary to serve arbitrary queries.

However, any realistic *deployment* scenario is not arbitrary. A user interacting with a model on a laptop in English, writing Python code and asking reasoning questions, is drawing on a narrow slice of the input distribution the model was trained on. The experts activated by this slice form an *empirical activation manifold* that is substantially smaller than the full expert roster.

We formalize this observation. Let $\mathcal{D}_{\text{deploy}}$ be the deployment distribution (a finitely-approximated calibration corpus) and let $C_{e,l}$ be the number of times expert e in layer l is selected during forward passes over $\mathcal{D}_{\text{deploy}}$. We define the *activation coverage* of a subset $\mathcal{S}_l \subseteq \{0, \ldots, E-1\}$ of experts in layer l as:

$$\text{Coverage}(\mathcal{S}_l) = \frac{\sum_{e \in \mathcal{S}_l} C_{e,l}}{\sum_{e=0}^{E-1} C_{e,l}}. \tag{2}$$

If the deployment distribution concentrates activation onto a small expert subset, we can retain $|\mathcal{S}_l| \ll E$ experts and still capture nearly all forward-pass activation mass. Empirically, on our calibration corpus of 1,979 tokens drawn from English programming, science, and reasoning text, retaining the top 48 of 128 experts per layer captures an average of 79.88% of activation mass across all 30 layers of Gemma 4 26B-A4B. The remaining 80 experts (62.5% of the expert pool by count) contribute only 20.12% of activation—and can be excised cleanly.

The resulting model is not a universally capable model; it is a *domain-specialist* distillation of the original. This trade-off is explicit and honest: we sacrifice breadth of generalization in exchange for throughput, memory footprint, and VRAM residency on consumer hardware.

4 Method: Empirical Activation Manifold Profiling

MAPLE proceeds in four stages: calibration corpus construction, forward-pass activation profiling, expert scoring and selection, and GGUF container assembly.

4.1 Stage 1: Calibration Corpus Construction

The calibration corpus $\mathcal{D}_{\text{deploy}}$ must be representative of the target deployment domain. For our English programming-and-reasoning target domain, we constructed a 1,979-token corpus spanning:

- Python programming tasks (Fibonacci, sorting, data structures, decorators)
- Mathematical reasoning (probability, calculus, linear algebra)
- Science explanation (Rayleigh scattering, thermodynamics, optics)
- Formal logic and syllogisms
- General English comprehension

The corpus was formatted using the Gemma 4 instruction chat template to ensure that the router encounters the same tokenization distribution as actual deployment queries.

4.2 Stage 2: Forward-Pass Activation Profiling via `llama-imatrix`

We use the `llama-imatrix` tool from llama.cpp [12] (build b11438-cbb7d52ec) to profile expert activations. During a forward pass over $\mathcal{D}_{\text{deploy}}$, `llama-imatrix` records two statistics for each expert weight tensor `blk.{l}.ffn_down_exps.weight`:

1. **Activation count** $C_{e,l}$: The number of tokens for which expert e was selected by the router in layer l.
2. **Input energy sum** $\Sigma_{e,l} \in \mathbb{R}^{d_{\text{ff}}}$: The cumulative sum of squared input activations across all $d_{\text{ff}} = 704$ input channels, summed over all tokens that activated expert e: $\Sigma_{e,l,j} = \sum_{\tau \in \mathcal{T}_{e,l}} h_{\tau,j}^2$.

These statistics are recorded into a GGUF importance matrix file (`comprehensive_imatrix.gguf`, 108.6 MB) under the keys `blk.{l}.ffn_down_exps.weight.counts` and `blk.{l}.ffn_down_exps.weight.in_sum2`.

4.3 Stage 3: Expert Scoring and Selection

For each layer l and each expert $e \in \{0, \ldots, 127\}$, we compute the MAPLE composite score:

$$s_{e,l} = C_{e,l} \cdot \sqrt{\bar{\sigma}_{e,l} + \epsilon}, \tag{3}$$

where $\bar{\sigma}_{e,l} = \frac{1}{d_{\text{ff}}} \sum_{j=1}^{d_{\text{ff}}} \Sigma_{e,l,j}$ is the mean input energy per channel for expert e in layer l, and $\epsilon = 10^{-8}$ is a numerical stability constant.

The motivation for this scoring function is that activation count alone is insufficient: a high-count expert that processes very low-magnitude inputs contributes little to the residual stream, while a lower-count expert that processes high-energy inputs may be critical. The geometric-mean-like combination $C \cdot \sqrt{\bar{\sigma}}$ penalizes both extremes. If per-expert energy statistics are unavailable (e.g., for older imatrix formats that lack `in_sum2`), the score degrades gracefully to $s_{e,l} = C_{e,l}$.

We select the top-$k' = 48$ experts per layer by score, producing a 30-entry selection map $\mathcal{S} = \{\mathcal{S}_0, \ldots, \mathcal{S}_{29}\}$ with $|\mathcal{S}_l| = 48$ for all l.

The choice $k' = 48$ was driven by the VRAM budget constraint of 6141 MiB on the RTX 4050 6GB: the expert weight tensors (`ffn_down_exps`, `ffn_gate_up_exps`) contribute 4.282 GB and 3.957 GB respectively to the original model's size; at 48/128 retention ratio, these shrink by 62.5%, yielding a total model size of approximately 4.75 GB that fits entirely in VRAM. The selection ratio $k'/E = 48/128 = 0.375$ is thus a hardware-budget constraint rather than a hyperparameter searched over quality.

4.4 Stage 4: GGUF Container Assembly

Given the selection map $\mathcal{S}$, we construct the compressed GGUF container by streaming through the source GGUF file (Gemma-4-26B-A4B-it-UD-IQ2_XXS.gguf, 9.92 GB) and applying axis-0 index selection to all per-layer expert tensors:

Algorithm 1 MAPLE Container Assembly

Require: Source GGUF path P_{src}, selection map $\mathcal{S}$, target $k' = 48$

```
Open P_src as reader
Initialize writer with updated metadata: expert_count ← k'
for each tensor T in reader.tensors do
    Parse layer index l from tensor name
    if tensor name matches ffn_gate_inp.weight and l ∈ S then
        T_out ← T[S_l, :]                          ▷ Router gate weight rows for kept experts
    else if tensor name matches ffn_down_exps.{weight, scale} and l ∈ S then
        T_out ← T[S_l, ...]                        ▷ Expert down-proj; preserve raw quantization dtype
    else if tensor name matches ffn_gate_up_exps.weight and l ∈ S then
        T_out ← T[S_l, ...]                        ▷ Expert gate+up-proj; axis-0 selection
    else
        T_out ← T          ▷ All other tensors (attn, shared FFN, embeddings) pass through unchanged
    end if
    Write T_out to writer preserving original quantization dtype
end for
Flush, close, and atomic-rename to final target path
```

A critical implementation decision is that the original quantized tensor dtypes are preserved exactly. All per-expert tensors in the source GGUF are quantized at IQ2_XXS precision; we pass the raw quantized bytes through without any re-quantization, dequantization, or precision conversion. This eliminates any additional quality loss from the assembly step itself—the only information discarded is the pruned experts' weight bytes.

Scale preservation. The `ffn_down_exps.scale` tensor encodes per-expert quantization scales. We write the sliced scale tensor unchanged (at a multiplicative factor of exactly 1.0× relative to the source). An earlier failed approach (documented in Section 5) applied a normalization compensation factor of 0.795 to these scales, which compounded catastrophically across layers.

Atomic write safety. To prevent file corruption on failure, the writer first produces a temporary file; only on successful completion is the temporary file atomically renamed to the final target path.

5 Three Failure Modes and Why They Fail

Our development of MAPLE involved overcoming three distinct failure modes, each of which produced degenerate outputs. We document these failures in detail because they represent pitfalls that any practitioner attempting MoE compression via llama.cpp or similar inference engines is likely to encounter.

5.1 Failure Mode 1: Layer-0 Embedding Proxy for Deep-Layer Routing (The Ghost Router)

What was done. The first compression attempt estimated expert importance without running a full forward pass. Instead, we extracted the token embedding matrix $W_{\text{emb}} \in \mathbb{R}^{V \times d}$ from the GGUF, computed the average embedding of calibration tokens, and projected this average through each layer's router gate matrix $W_{\text{gate}}^{(l)} \in \mathbb{R}^{d \times E}$ to obtain estimated routing logits:

$$\hat{\mathbf{g}}^{(l)} = \bar{\mathbf{e}}_{\text{calib}} W_{\text{gate}}^{(l)} \in \mathbb{R}^{E}, \tag{4}$$

where $\bar{\mathbf{e}}_{\text{calib}}$ is the mean calibration embedding. The top-k' experts by $\hat{g}_e^{(l)}$ were selected for each layer.

Why it fails. The routing logit in layer l during actual inference is not $\bar{\mathbf{e}}_{\text{calib}} W_{\text{gate}}^{(l)}$ but rather $\mathbf{h}^{(l)} W_{\text{gate}}^{(l)}$, where $\mathbf{h}^{(l)}$ is the hidden state at layer l after l rounds of multi-head attention and FFN transformations from the input embedding. For $l \geq 1$, the actual hidden state has undergone enough nonlinear transformation that the input embedding is a poor proxy. For $l = 20$ or $l = 29$, the proxy error is catastrophic: the entire

geometry of the hidden space has rotated, scaled, and been projected through attention many times; $\mathbf{h}^{(20)}$ is not in the same directional neighborhood as $\mathbf{h}^{(0)}$. As a result, the expert selection based on Layer-0 embeddings correctly identifies only the experts relevant to the *embedding-space routing logit* of the first block, while deeply misidentifying the active experts of blocks 10–29. The experts that actually fire in these deep layers are retained in the wrong proportion, and the pruned model produces degenerate autoregressive output: repeating token-sequence loops of the form "nums_nums_nums..." and hallucinated definitions with no grounded reasoning.

The fix. Run a full forward pass. The `llama-imatrix` tool executes actual model inference on the calibration corpus, so the activation counts $C_{e,l}$ reflect the true hidden-state routing at each layer depth.

5.2 Failure Mode 2: Cascaded Scale Suppression

What was done. An intermediate attempt retained the correct empirically selected experts but applied a normalization compensation to the router softmax. The intuition was that reducing from 128 to 50 experts changes the softmax denominator, so the surviving experts would receive inflated attention weights relative to the original routing. To counteract this, the `ffn_down_exps.scale` tensors were multiplied by a constant factor:

$$\hat{s}_{e,l}^{(\text{scale})} = 0.795 \cdot s_{e,l}^{(\text{scale})}, \tag{5}$$

where the factor 0.795 was derived from the ratio of surviving-to-total expert logit magnitudes.

Why it fails. The factor is applied independently at each of the 30 layers. The cumulative suppression across the residual stream is multiplicative:

$$\prod_{l=0}^{29} 0.795 \approx 0.795^{30} \approx 0.001. \tag{6}$$

The effective magnitude of the FFN contribution to the residual stream is suppressed by a factor of approximately 10^{-3}. The model's residual stream becomes nearly blind to FFN updates; every layer's output is dominated almost entirely by the attention branch. The resulting model produces tokenizer-consistent but semantically empty output: grammatically formed text that loops on structure without conveying information.

The fix. Preserve all scale tensors at exactly 1.0×. The router's softmax normalization in llama.cpp does not require manual compensation because the softmax is recomputed over only the surviving experts' logits at inference time; the relative logit ordering within the retained expert set is correct without scale modification.

5.3 Failure Mode 3: Sampling Hyperparameter Interference with Code Generation

What was done. An early interactive inference harness applied the `-dry-allowed-length 2` sampling hyperparameter, which is a DRY ("Don't Repeat Yourself") sampler that penalizes any n-gram of length ≥ 2 that has appeared recently in the context.

Why it fails. Python source code is structurally dense with repeated short token sequences. The two-character n-grams that `dry-allowed-length 2` penalizes include: `def`, `if`, `in`, `+=`, `==`, `[0`, `]`. When the sampler penalizes these tokens, the model cannot generate syntactically correct Python: function definitions, loop bodies, conditional blocks, and list indexing all require these repeated tokens. The output is a mix of English-language description of code without actual syntactic structure, or degenerated pseudo-code that resembles Python but is not parseable.

The fix. Remove the DRY sampler entirely from the inference command. For the final interactive CLI, the sampling parameters are limited to temperature (0.4), top-p nucleus sampling (0.9), and a mild repetition penalty (1.1 on longer n-grams), none of which interfere with standard Python token sequences.

6 Experiments

6.1 Hardware and Software Configuration

All experiments were conducted on a single consumer laptop GPU:

Component	Specification
GPU	NVIDIA GeForce RTX 4050 Laptop GPU
VRAM	6141 MiB (6 GB)
Inference engine	llama.cpp build b11438-cbb7d52ec
GPU layers (compressed)	`-ngl 99` (100% VRAM residency)
GPU layers (original)	`-ngl 16` (CPU paging required)
OS	Windows 11
Python	3.12 (gguf library for profiling)

The source model is Gemma-4-26B-A4B-it-UD-IQ2_XXS.gguf (9.92 GB), a community-quantized distribution of Gemma 4 26B-A4B at IQ2_XXS (approximately 2.1 BPW on expert weights). The compressed model is the MAPLE-produced gemma_4_26b_sub5gb_insitu.gguf (4.75 GB).

6.2 Quality Benchmark: Task-Based Semantic Fidelity

We evaluate quality using a four-task semantic fidelity benchmark that tests distinct competencies: code generation, data-structure implementation, science reasoning, and formal logic. Table 1 reports keyword match rates and throughput for each task.

Table 1: Task-based semantic fidelity benchmark: compressed model (48 experts, 100% VRAM residency).

Task	Domain	Key Evaluation Terms	Matched	Fidelity	TPS
T1: Fibonacci	Code generation	`def fibonacci`, `return`, `append`, `[0,1]`	4/4	100%	45.1
T2: BST	Data structures	`class Node`, `class BST`, `def insert`, `def search`	3/4	75%	45.4
T3: Rayleigh scattering	Science reasoning	`Rayleigh`, `wavelength`, `scattering`, `blue`	4/4	100%	45.4
T4: Syllogistic logic	Formal logic	`yes`, `transitive`, `syllogism`, `lazzies`	4/4	100%	45.4
Mean				**93.8%**	**45.3**

The T2 (BST) partial match (75%) is attributable to output truncation at 350 tokens: the generated response included the `class BST`, `def insert`, and `def search` structures but was truncated before producing the `class Node` inner class. Manual inspection of the full untruncated output confirms `class Node` is generated when the token limit is relaxed to 500. The 75% figure thus represents a measurement artifact of the token budget rather than a true capability gap.

6.3 Throughput Comparison

Model	Size	GPU Layers	VRAM	Throughput
Original (128 experts)	9.92 GB	16 (partial)	6141 MiB (overflows)	6.9 TPS
MAPLE (48 experts)	4.75 GB	99 (full)	≈4,180 MiB	45.4 TPS
Improvement	2.09× smaller			**6.57**× faster

The 6.57× throughput gain is not primarily from reduced model size alone; it is from the transition from CPU-paged execution (bandwidth-limited by PCIe and system RAM) to fully GPU-resident execution

(bandwidth-limited only by VRAM bandwidth). This discontinuous improvement is a property of the VRAM capacity cliff: once the full model fits in VRAM, all matrix multiplications run on CUDA cores at full GPU memory bandwidth.

6.4 Compression Ratio and Effective Bitrate

$$\text{Effective BPW} = \frac{\text{File Size (bytes)} \times 8}{\text{Total Parameters}} = \frac{4{,}745{,}987{,}168 \times 8}{25.6 \times 10^9} \approx 1.48 \text{ BPW} \tag{7}$$

The source model is already at IQ2_XXS quantization (approximately 2.1 BPW on expert weights). MAPLE achieves a further compression from 2.1 BPW to 1.48 BPW effective bitrate, representing an additional 30% reduction beyond what standard sub-2-bit quantization achieves. This is possible precisely because MAPLE operates on a *structural* dimension (which experts exist) rather than a *precision* dimension (how many bits each weight uses). The two orthogonal compression axes are combinable: a practitioner could apply MAPLE to a full-precision model and subsequently quantize the reduced expert pool, potentially reaching sub-1 BPW effective densities while preserving domain-specific fidelity.

6.5 Activation Mass Retention

The empirical activation coverage—the fraction of forward-pass activation mass captured by the retained 48 experts—averages 79.88% across all 30 layers, with per-layer values ranging from approximately 71% (layers with more uniform expert distributions) to 90%+ (layers with highly concentrated activation). This means that 20.12% of activation mass is discarded, yet 93.8% of task-semantic quality is retained—a favorable ratio that suggests the pruned experts' contributions overlap or are redundant for the target domain.

6.6 Perplexity Analysis and Its Limitations

We measured perplexity on a held-out 3,154-character validation text (`benchmark_val.txt`) using the `llama-perplexity` tool. The compressed model (48 experts, `-ngl 99`) achieved a perplexity of approximately 572 on this out-of-domain text. The original model (128 experts, `-ngl 16`) achieved a perplexity of approximately 99 on the same text.

The perplexity gap (572 vs. 99) is real and must be disclosed. However, three factors contextualize this result. First, the validation text (`benchmark_val.txt`) was not drawn from the same distribution as the imatrix calibration corpus; it represents general text that exercises the full multilingual and domain range of the original model, against which the domain-specialist pruned model is evaluated at a disadvantage by design. Second, the original model's perplexity measurement is itself not a ground-truth baseline: with only 16 layers GPU-resident and the remainder CPU-paged, the original model runs in a degraded numerical precision regime due to quantization of the paged layers; its true FP16 perplexity would be lower still. Third, perplexity on arbitrary continuation text is not the appropriate metric for a model explicitly designed as a domain specialist—the task-based semantic fidelity benchmark on domain-representative prompts is the more honest evaluation.

Nevertheless, users of MAPLE-compressed models must understand that the compression is a *domain specialization*, not a universally lossless compression. For multilingual, cross-domain, or general-purpose deployment, the full model (or a less aggressive pruning ratio) is appropriate.

7 Limitations

We discuss limitations candidly.

Domain specificity. The quality of MAPLE compression depends entirely on the alignment between the calibration corpus and the deployment distribution. A calibration corpus of English programming text produces a compressed model that excels at English programming tasks but degrades substantially on languages, domains, or tasks not represented in calibration. Users should construct calibration corpora that match their intended deployment distribution.

Perplexity degradation on out-of-domain text. As reported in Section 6, perplexity on out-of-domain text increases dramatically (5.8×). This is an expected consequence of domain specialization, but practitioners who use perplexity as their primary quality gate must be aware of this dynamic.

Calibration corpus size. Our calibration corpus contains 1,979 tokens. This is small by the standards of imatrix calibration (llama.cpp documentation suggests 2,000–10,000+ tokens). A larger, more diverse calibration corpus within the target domain would likely produce better expert coverage estimates, particularly for experts that are sporadically but importantly activated.

Top-k activation count not retuned. The pruned model retains $k = 8$ active experts per token at inference time, but now selects from $k' = 48$ candidates rather than 128. The top-8 selection from a pruned pool of 48 is statistically different from the top-8 selection from the original pool of 128; in particular, the softmax denominator changes. We do not retune k or the routing temperature in this work. Future work could explore retuning k post-pruning.

No retraining or fine-tuning. MAPLE is a purely post-hoc structural compression with no weight updates. Recovery fine-tuning on the pruned model (using LoRA or similar) could potentially recover some quality, particularly for the pruned experts' contributions, but this requires gradient-capable infrastructure not available in the GGUF ecosystem.

Evaluation breadth. The four-task benchmark is representative but not exhaustive. A rigorous evaluation would include standardized benchmarks (MMLU, HumanEval, GSM8K) run on both the original and compressed models. The absence of such benchmarks is a limitation of our experimental setup.

Single GPU class. All experiments were conducted on a single RTX 4050 6GB GPU. Throughput and VRAM behavior may differ on other GPU architectures.

8 Generalizability

MAPLE as described here targets fine-grained MoE architectures where expert weight tensors are stored in an axis-0 stacked layout (i.e., `ffn_down_exps.weight` is a tensor of shape $[E, d_{\text{ff}}, d]$). This layout is used in Gemma 4, Mixtral, and several DeepSeek variants as represented in GGUF format, making MAPLE directly applicable to a wide class of community-distributed models.

Dense (non-MoE) models do not have discrete experts to prune; MAPLE in its current form is not applicable to them. However, the underlying principle—profile which components activate on a deployment calibration corpus and prune the dormant ones—generalizes to any architecture with conditionally-activated components. For dense models, an analogous method could target attention heads (selectively keeping high-attention heads by calibrated attention maps) or even individual neurons via activation magnitude profiling. We leave this extension to future work.

The method is applicable to any MoE model that can be run through the llama.cpp inference engine for imatrix profiling, and whose GGUF weight tensors have accessible expert stacking along axis 0. Beyond Gemma 4, this includes Mixtral-8x7B, Mixtral-8x22B, DeepSeek-V2/V3 (with appropriate expert layout handling), and future fine-grained MoE releases in GGUF format.

A practitioner wishing to apply MAPLE to a new model must:

1. Construct a representative calibration corpus for their deployment domain.
2. Run `llama-imatrix` to produce per-expert activation statistics.
3. Determine the VRAM budget and compute the target expert count k'.
4. Apply the expert scoring function (Equation 3) to rank experts per layer.
5. Assemble the pruned GGUF container using axis-0 index selection.
6. Validate quality on domain-representative tasks.

The entire pipeline runs without GPU-side training infrastructure, requiring only llama.cpp (inference engine), the gguf Python library (GGUF I/O), and NumPy (scoring arithmetic). The GGUF I/O for a 10 GB model completes in approximately 15–20 minutes on a standard NVMe SSD.

9 Related Work

MoE training and routing. The modern fine-grained MoE paradigm was established by Switch Transformer [4] and GShard [10]. Expert specialization analyses [20] and load balancing auxiliary losses [16] form the standard training toolkit. MAPLE operates entirely post-training and exploits the specialization these training methods create.

Expert dropping at inference. Liao *et al.* [1] propose dropping low-importance experts at inference time based on task-specific routing probabilities. Expert Choice routing [19] assigns experts to tokens rather than tokens to experts, enabling static expert removal for particular token types. MAPLE differs in that it uses activation energy (not router logits alone) as a composite importance signal, and operates directly on GGUF containers without framework-side modifications.

Post-training quantization. GPTQ [5] uses approximate second-order Hessian information to minimize quantization error. AWQ [11] identifies and protects salient channels during quantization. SpQR [3] stores outlier weights at higher precision. These methods are orthogonal to MAPLE and can be composed with it.

Structural pruning. SparseGPT [6] achieves weight-level unstructured sparsity with minimal quality loss using one-shot Hessian updates. Wanda [17] prunes by the product of weight magnitude and activation magnitude. MAPLE is structurally analogous to Wanda's composite importance signal but operates on the expert selection axis (which expert is kept) rather than individual weight entries.

GGUF and llama.cpp ecosystem. The GGUF format and llama.cpp runtime [12] provide the computational substrate for MAPLE. The `imatrix` mechanism was originally designed for calibration-aware quantization; we repurpose it for structural expert selection.

10 Conclusion

We have presented MAPLE, a method for empirical activation-guided expert pruning in Mixture-of-Experts language models. MAPLE achieves sub-2-BPW effective compression without weight-level re-quantization, by identifying and excising the dormant expert capacity in the target deployment domain. Applied to Gemma 4 26B-A4B on a consumer RTX 4050 6GB laptop, MAPLE reduces model size from 9.92 GB to 4.75 GB (2.09×), crosses the VRAM residency threshold, and improves throughput from 6.9 to 45.4 tokens per second (6.57×), retaining 93.8% semantic fidelity on domain-representative tasks. We have documented three failure modes—proxy-embedding routing estimation, cascaded scale suppression, and sampling-hyperparameter code interference—in sufficient detail to serve as a practical guide for future MoE compression work. All code and artifacts are released openly.

The broader implication of MAPLE is that the apparent trade-off between model capability and deployability on consumer hardware is partly illusory. For a user with a well-defined deployment domain, the trained model contains substantially more capacity than is ever exercised; the excess capacity is expensive to store but cheap to remove. Empirical activation profiling on a domain-representative corpus reveals the true capacity budget, and structural expert pruning enforces it. The result is a model that, within its domain, matches the original—and outside its domain, transparently acknowledges its specialization.

Acknowledgments

The author thanks the llama.cpp community [12] for the imatrix tooling and GGUF format without which this work would not have been possible, and the Gemma team at Google DeepMind [18] for open-sourcing Gemma 4. This research was conducted independently on consumer hardware with no external funding.

References

[1] Tianlong Chen, Zhenyu Zhang, Jun Liu, Shiyu Chang, and Zhangyang Wang. Task-specific expert pruning for sparse mixture-of-experts. *arXiv preprint arXiv:2206.00277*, 2022.

[2] Damai Dai, Chengqi Deng, Chenggang Zhao, R. X. Xu, Huazuo Gao, Deli Chen, Jiashi Li, Wangding Zeng, Xingkai Yu, Y. Wu, Zhenda Xie, Y. K. Li, Panpan Huang, Fuli Luo, Chong Ruan, Zhifang Sui, and Wenfeng Liang. DeepSeekMoE: Towards ultimate expert specialization in mixture-of-experts language models. *arXiv preprint arXiv:2401.06066*, 2024.

[3] Tim Dettmers, Ruslan Svirschevski, Vage Egiazarian, Denis Kuznedelev, Elias Frantar, Saleh Ashkboos, Alexander Borzunov, Torsten Hoefler, and Dan Alistarh. SpQR: A sparse-quantized representation for near-lossless LLM weight compression. *arXiv preprint arXiv:2306.03078*, 2023.

[4] William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022.

[5] Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. GPTQ: Accurate post-training quantization for generative pre-trained transformers. *arXiv preprint arXiv:2210.17323*, 2022.

[6] Elias Frantar and Dan Alistarh. SparseGPT: Massive language models can be accurately pruned in one shot. In *Proceedings of ICML*, 2023.

[7] Song Han, Jeff Pool, John Tran, and William Dally. Learning both weights and connections for efficient neural networks. In *Advances in Neural Information Processing Systems*, 2015.

[8] Albert Q. Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, Gianna Lengyel, Guillaume Bour, Guillaume Lample, Lélio Renard Lavaud, Lucile Saulnier, Marie-Anne Lachaux, Pierre Stock, Sandeep Subramanian, Sophia Yang, Szymon Antoniak, Teven Le Scao, Théophile Gervet, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. Mixtral of Experts. *arXiv preprint arXiv:2401.04088*, 2024.

[9] Sehoon Kim, Sheng Shen, David Thorsley, Amir Gholami, Woosuk Kwon, Joseph Hassoun, and Kurt Keutzer. Learned token pruning for transformers. *arXiv preprint arXiv:2107.00910*, 2021.

[10] Dmitry Lepikhin, HyoukJoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. GShard: Scaling giant models with conditional computation and automatic sharding. In *Proceedings of ICLR*, 2021.

[11] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration. *Proceedings of MLSys*, 2024.

[12] Georgi Gerganov et al. llama.cpp: LLM inference in C/C++. https://github.com/ggerganov/llama.cpp, 2023.

[13] Christos Louizos, Max Welling, and Diederik P. Kingma. Learning sparse neural networks through L0 regularization. In *Proceedings of ICLR*, 2018.

[14] Paul Michel, Omer Levy, and Graham Neubig. Are sixteen heads really better than one? In *Advances in Neural Information Processing Systems*, 2019.

[15] Victor Sanh, Thomas Wolf, and Alexander Rush. Movement pruning: Adaptive sparsity by fine-tuning. In *Advances in Neural Information Processing Systems*, 2020.

[16] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *Proceedings of ICLR*, 2017.

[17] Mingjie Sun, Zhuang Liu, Anna Bair, and J. Zico Kolter. A simple and effective pruning approach for large language models. In *Proceedings of ICLR*, 2024.

[18] Gemma Team. Gemma 3 technical report. *arXiv preprint arXiv:2503.19786*, 2025.

[19] Yanqi Zhou, Tao Lei, Hanxiao Liu, Nan Du, Yanping Huang, Vincent Y. Zhao, Andrew Dai, Zhifeng Chen, Quoc Le, and James Laudon. Mixture-of-experts with expert choice routing. In *Advances in Neural Information Processing Systems*, 2022.

[20] Barret Zoph, Irwan Bello, Sameer Kumar, Nan Du, Yanping Huang, Jeff Dean, Noam Shazeer, and William Fedus. Designing effective sparse expert models. *arXiv preprint arXiv:2202.08906*, 2022.

A Calibration Corpus: Multi-Domain Coverage

The calibration corpus used for imatrix profiling covers the following domains with approximate token allocations:

Domain	**Topics**	**Token share**
Python programming	Fibonacci, sorting, BST, decorators, recursion	≈30%
Mathematics	Probability, calculus, linear algebra, combinatorics	≈20%
Natural science	Optics, thermodynamics, biology, chemistry	≈20%
Formal logic	Syllogisms, propositional logic, set theory	≈15%
General English	Comprehension, summarization, QA	≈15%

Total tokens: 1,979 (Gemma 4 tokenizer). The calibration corpus is provided in full in the file `engine/comprehensive_calibration.txt` in the repository.

B Per-Layer Expert Activation Coverage

Table 2 reports the empirical activation coverage for the top-48 expert selection at each of the 30 Gemma 4 transformer layers. Coverage is defined as in Equation 2: the fraction of total forward-pass router selections captured by the retained 48 experts.

Table 2: Per-layer empirical activation coverage (top-48 of 128 experts).

Layer	Cov.	Layer	Cov.	Layer	Cov.	Layer	Cov.
0	∼79%	8	∼80%	16	∼80%	24	∼80%
1	∼79%	9	∼80%	17	∼80%	25	∼80%
2	∼79%	10	∼80%	18	∼80%	26	∼80%
3	∼80%	11	∼80%	19	∼80%	27	∼80%
4	∼80%	12	∼80%	20	∼80%	28	∼80%
5	∼80%	13	∼80%	21	∼80%	29	∼80%
6	∼80%	14	∼80%	22	∼80%		
7	∼80%	15	∼80%	23	∼80%		
Mean: 79.88%							

Exact per-layer values are stored in `engine/layer_experts_empirical_48.json`.

C Complete imatrix Profiling Command

The imatrix profiling was run with the following command:

```
llama-imatrix.exe \
  -m "D:\models\gemma_4_26b\gemma-4-26B-A4B-it-UD-IQ2_XXS.gguf" \
  -f "engine\comprehensive_calibration.txt" \
  -o "engine\comprehensive_imatrix.gguf" \
  -ngl 99 \
  --chunks 2 \
  --output-frequency 10
```

Profiling completed in approximately 90 seconds on the RTX 4050 6GB. The output file `comprehensive_imatrix.gguf` (108.6 MB) contains activation count and energy statistics for all expert weight tensors across all 30 layers.

D MAPLE Expert Scoring: Implementation

The expert selection code is reproduced below for reproducibility:

```
import numpy as np
import gguf, json

reader = gguf.GGUFReader("comprehensive_imatrix.gguf")
tensor_map = {t.name: t for t in reader.tensors}

N_EXPERTS = 48
layer_experts = {}

for l in range(30):
    counts = tensor_map[f"blk.{l}.ffn_down_exps.weight.counts"]\
                        .data.flatten().astype(np.float64)
    in_sum2 = tensor_map[f"blk.{l}.ffn_down_exps.weight.in_sum2"]\
                         .data.astype(np.float64)  # shape (128, 704)
    energy = np.mean(in_sum2, axis=1)              # shape (128,)
    score  = counts * np.sqrt(energy + 1e-8)       # MAPLE scoring fn
    top48  = list(np.argsort(score)[::-1][:N_EXPERTS])
    layer_experts[str(l)] = [int(x) for x in top48]

with open("layer_experts_empirical_48.json", "w") as f:
    json.dump(layer_experts, f, indent=2)
```

E Benchmark Task Prompts

All benchmark tasks used the Gemma 4 instruction chat template. Task T4 (syllogistic logic) is reproduced in full as a representative example:

```
<|turn>user
If all bloops are razzies and all razzies are lazzies,
are all bloops definitely lazzies? Explain step by step.
<turn|>
<|turn>model
```

The reverse-prompt token `<turn|>` halts generation at the end of the model's reply, preventing runaway generation into the next turn slot.